test on prod. break nothing.

mori spins up an ephemeral proxy to your production database — full read access, sandboxed writes, automatic teardown. run your migrations, test your queries, and validate schema changes against real data without touching a single prod row.

brew install mori

or let your AI agent set it up

$ mori start --provider Cloud SQL --db Cloud SQL

under the hood

STEP 1

Connect to prod.

point mori at your production database. it authenticates with your existing provider credentials — no config, no IAM headaches.

STEP 2

Run all your queries.

mori creates a copy-on-write proxy layer. reads hit real data, writes are sandboxed to an ephemeral instance. prod untouched.

STEP 3

Test, then tear it down.

run migrations, seed data, break things. when you're done, the instance dies and nothing persists. memento mori.

features

01 **zero-config provider auth**

auto-detects credentials for AWS RDS, GCP Cloud SQL, Supabase, Neon, PlanetScale, etc — or bring your own connection string.

02 **copy-on-write isolation**

reads from prod, writes to ephemeral sandbox. you and your agents get live production data, and the confidence that your tests will never mutate your database.

03 **migration testing**

validate schema changes by running your migrations against a copy of real data before deploying. let that intern drop the unique constraint on emails — who cares?

01 **model context protocol**

exposes the mori proxy as an mcp server out-of-the-box. drop it into claude code or cursor, and help your agent write more intelligent code with better validation.

01 **tui dashboard support**

run in headless mode for pipeline integration. just replace

moridb.sh

built with love in los angeles, california

psrth.sh GitHub Email X/Twitter